

# ITIS4260 Project Machine Learning 3

April 30, 2025

## 1 Project Machine Learning 3

ITIS 4260 Efren Antonio

```
[2]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import confusion_matrix, roc_curve, roc_auc_score, \
    accuracy_score
```

```
[3]: # Read in the data from the CSV file
df = pd.read_csv('datasets/wc1.csv')
pd.set_option('display.max_columns', None)
df.head()
```

```
[3]:
```

|   | inline_count | external_count | onclick_count | onload_count | onchange_count | \ |
|---|--------------|----------------|---------------|--------------|----------------|---|
| 0 | 21.0         | 23.0           | 1             | 131.0        | 0.0            |   |
| 1 | 13.0         | 30.0           | 2             | 4.0          | 1.0            |   |
| 2 | 0.0          | 3.0            | 1             | 0.0          | 0.0            |   |
| 3 | 21.0         | 11.0           | 1             | 10.0         | 1.0            |   |
| 4 | 10.0         | 5.0            | 1             | 0.0          | 0.0            |   |

|   | avg_inline_script_block | avg_external_script_block | avg_onclick_count | \ |
|---|-------------------------|---------------------------|-------------------|---|
| 0 | 0.0                     | 662.062500                | 30737             |   |
| 1 | 0.0                     | 55.777778                 | 2951              |   |
| 2 | 0.0                     | 207.000000                | 387               |   |
| 3 | 0.0                     | 104.800000                | 1505              |   |
| 4 | 0.0                     | 473.000000                | 4121              |   |

|   | avg_onload_count | avg_onchange_count | avg_cyc_complexity | \ |
|---|------------------|--------------------|--------------------|---|
| 0 | 4                | 16                 | 0.812500           |   |
| 1 | 12               | 18                 | 0.555556           |   |
| 2 | 2                | 1                  | 1.000000           |   |

|   |   |   |          |
|---|---|---|----------|
| 3 | 2 | 5 | 0.400000 |
| 4 | 2 | 3 | 1.000000 |

|   | library_code_count | type  |
|---|--------------------|-------|
| 0 | 13                 | alexa |
| 1 | 10                 | alexa |
| 2 | 1                  | phish |
| 3 | 2                  | alexa |
| 4 | 3                  | alexa |

```
[4]: # Label encoding
df['type_phish'] = (df['type'] == 'phish').astype(int)
df.drop(columns='type', inplace=True)

# Normalize data
cols_to_normalize = [
    'inline_count', 'external_count', 'onclick_count', 'onload_count',
    'onchange_count', 'avg_inline_script_block', 'avg_external_script_block',
    'avg_onclick_count', 'avg_onload_count', 'avg_onchange_count',
    'avg_cyc_complexity', 'library_code_count'
]
scaler = MinMaxScaler()
df[cols_to_normalize] = scaler.fit_transform(df[cols_to_normalize])

# Features and target
X = df.drop(columns=['type_phish'])
y = df['type_phish']

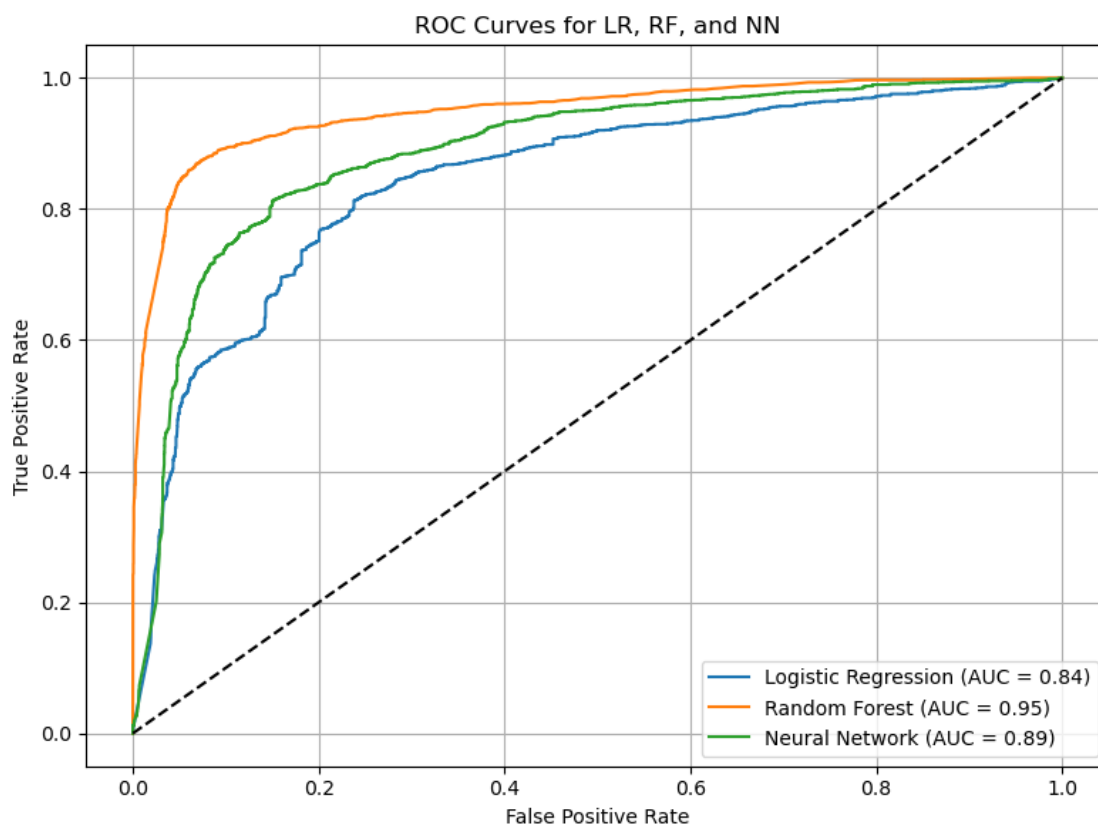
# Split for initial ROC curves
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,
    random_state=42)
```

```
[5]: # Logistic Regression
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)
lr_probs = lr.predict_proba(X_test)[: , 1]
fpr_lr, tpr_lr, _ = roc_curve(y_test, lr_probs)
auc_lr = roc_auc_score(y_test, lr_probs)
```

```
[6]: # Random Forest
rf = RandomForestClassifier(n_estimators=1000)
rf.fit(X_train, y_train)
rf_probs = rf.predict_proba(X_test)[: , 1]
fpr_rf, tpr_rf, _ = roc_curve(y_test, rf_probs)
auc_rf = roc_auc_score(y_test, rf_probs)
```

```
[7]: # Neural Network
nn = MLPClassifier(max_iter=1000)
nn.fit(X_train, y_train)
nn_probs = nn.predict_proba(X_test)[: , 1]
fpr_nn, tpr_nn, _ = roc_curve(y_test, nn_probs)
auc_nn = roc_auc_score(y_test, nn_probs)
```

```
[8]: # Plot ROC curves
plt.figure(figsize=(8, 6))
plt.plot(fpr_lr, tpr_lr, label=f'Logistic Regression (AUC = {auc_lr:.2f})')
plt.plot(fpr_rf, tpr_rf, label=f'Random Forest (AUC = {auc_rf:.2f})')
plt.plot(fpr_nn, tpr_nn, label=f'Neural Network (AUC = {auc_nn:.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves for LR, RF, and NN")
plt.legend(loc="lower right")
plt.grid(True)
plt.tight_layout()
plt.show()
```



```
[9]: # Random Forest gives the best ROC performance with AUC 0.95, followed by  
      ↪ Neural Network (~0.90), and Logistic Regression (~0.85).
```

```
[10]: # Setup K-Fold cross-validation  
cv = KFold(n_splits=10, shuffle=True, random_state=42)  
  
# Define models  
lr_model = LogisticRegression(max_iter=1000, random_state=42)  
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)  
nn_model = MLPClassifier(hidden_layer_sizes=(50,), max_iter=300,  
      ↪ random_state=42)  
  
# Run cross-validation  
lr_scores = cross_val_score(lr_model, X, y, cv=cv, scoring='accuracy',  
      ↪ n_jobs=-1)  
rf_scores = cross_val_score(rf_model, X, y, cv=cv, scoring='accuracy',  
      ↪ n_jobs=-1)  
nn_scores = cross_val_score(nn_model, X, y, cv=cv, scoring='accuracy',  
      ↪ n_jobs=-1)
```

```
[11]: # Print CV mean scores  
print("10-Fold CV Accuracy:")  
print(f"Logistic Regression: {lr_scores.mean():.4f}")  
print(f"Random Forest:      {rf_scores.mean():.4f}")  
print(f"Neural Network:      {nn_scores.mean():.4f}")
```

```
10-Fold CV Accuracy:  
Logistic Regression: 0.8239  
Random Forest:      0.9267  
Neural Network:     0.8597
```

```
[12]: # Random Forest shows the highest average accuracy, indicating best performance  
      ↪ under cross-validation.
```

```
[13]: # Normalize features using MinMaxScaler  
scaler = MinMaxScaler()  
X_scaled = scaler.fit_transform(X)  
# Split again after scaling  
X_train_n, X_test_n, y_train_n, y_test_n = train_test_split(X_scaled, y,  
      ↪ test_size=0.33, random_state=133)
```

```
[14]: # Train with normalized RF  
rf_n = RandomForestClassifier(n_estimators=100, random_state=42)  
rf_n.fit(X_train_n, y_train_n)  
auc_rf_n = roc_auc_score(y_test_n, rf_n.predict_proba(X_test_n)[:, 1])  
  
# Train with normalized NN  
nn_n = MLPClassifier(max_iter=500, random_state=1)
```

```

nn_n.fit(X_train_n, y_train_n)
auc_nn_n = roc_auc_score(y_test_n, nn_n.predict_proba(X_test_n)[:, 1])

# Train with normalized LR
lr_n = LogisticRegression(max_iter=10000)
lr_n.fit(X_train_n, y_train_n)
auc_lr_n = roc_auc_score(y_test_n, lr_n.predict_proba(X_test_n)[:, 1])

```

```

[15]: print("\nAUC after normalization:")
      print(f"Logistic Regression: {auc_lr_n:.4f}")
      print(f"Random Forest:      {auc_rf_n:.4f}")
      print(f"Neural Network:      {auc_nn_n:.4f}")

```

```

AUC after normalization:
Logistic Regression: 0.8377
Random Forest:      0.9493
Neural Network:     0.8883

```

```

[16]: # Normalization helps Neural Network and Logistic Regression. Random Forest is
      ↪ less sensitive to it.

```